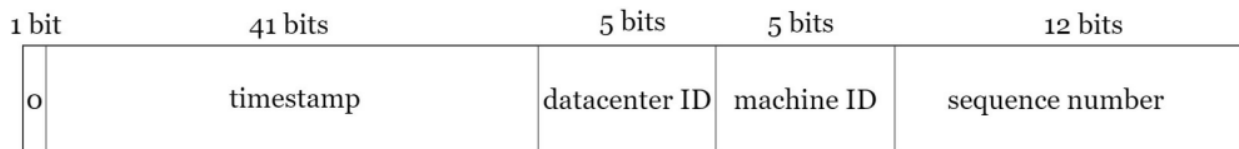


# Design a Unique ID Generator - Step 3: Design Deep Dive

## 1. Approach Chosen

- Based on **Snowflake design** by Twitter



## 2. ID Components Behavior

### Datacenter ID & Machine ID

- Chosen at **startup time**
- Remain **fixed during system runtime**
- Any change requires **careful review**

Reason:

- Incorrect changes can cause **ID conflicts**

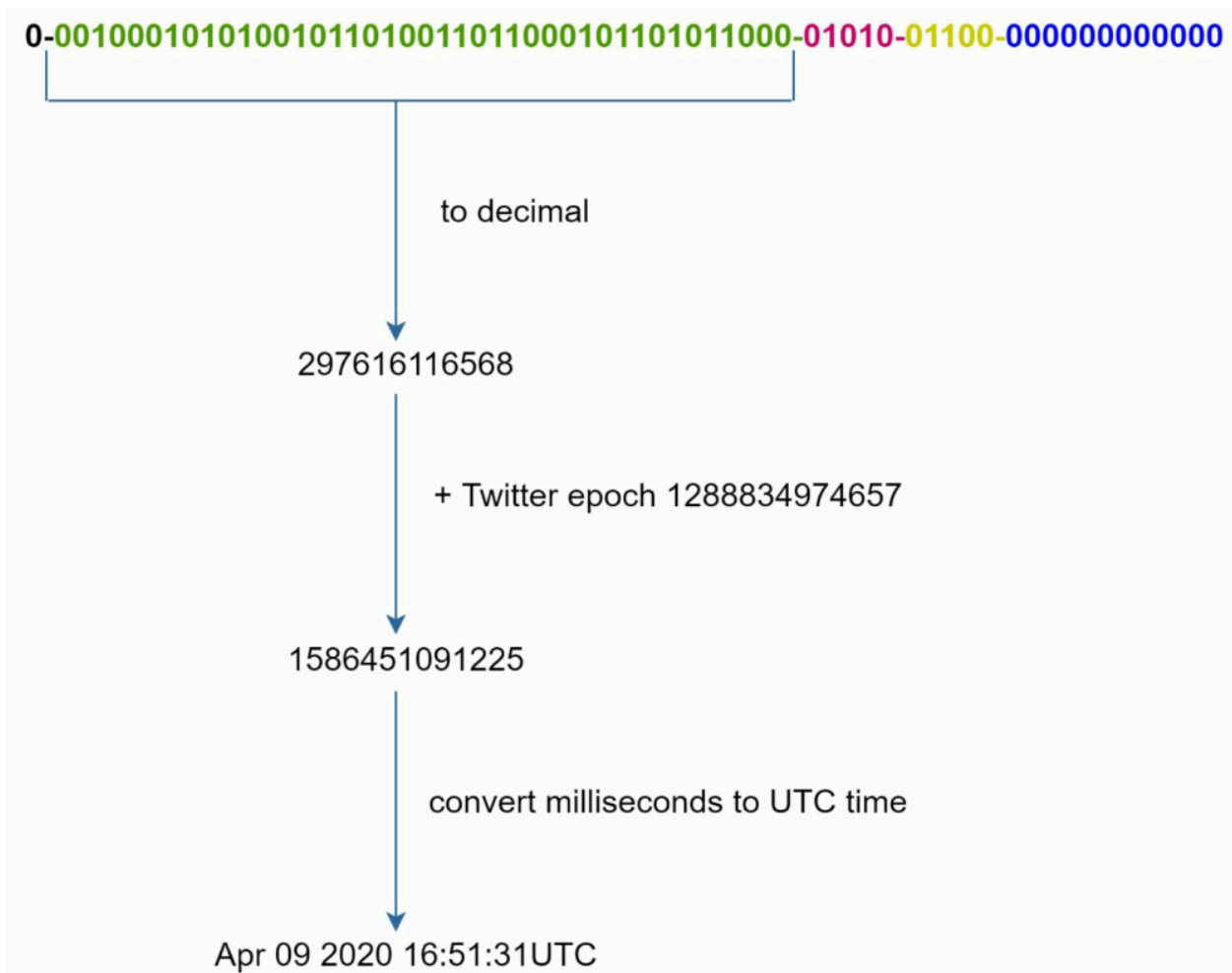
### Runtime Generated Fields

- **Timestamp**
- **Sequence Number**

### 3. Timestamp (41 bits)

#### Key Points

- Most important part of the ID
- Represents **time in milliseconds**
- Ensures **IDs are sortable**



#### Maximum Time Supported

None

$2^{41} - 1 = 219902325551 \text{ ms}$

## Conversion

None

$\approx 69 \text{ years}$

## Implications

- System works for **~69 years**
- Using a **recent custom epoch** delays overflow
- After 69 years:
  - Define a new epoch OR
  - Migrate to a new ID system

## 4. Sequence Number (12 bits)

### Capacity

None

$2^{12} = 4096 \text{ IDs per millisecond per machine}$

### Behavior

- Starts from **0 every millisecond**

- Increments for each new ID
- Resets when the millisecond changes

## Key Insight

- Only used when **multiple IDs are generated within same millisecond**

## 5. Throughput

Per machine:

None

Max = 4096 IDs / millisecond

## 6. Key Takeaway

Combination ensures:

- **Timestamp** → ordering
- **Datacenter + Machine ID** → distributed uniqueness
- **Sequence** → high throughput